SISTEMA DE APOIO AO DIAGNÓSTICO DE OBESIDADE

FIAP - TECH CHALLENGE FASE 04
DATA ANALYTICS (10DTAT)

TECH CHALLENGE

Fase 04

ANDERSON CORREIA BALBINO. RM 365550

ELECIDES TEIXEIRA JUNIOR. RM 366315

GABRIEL MACIEL DOS SANTOS. RM 365687

RAFAEL VICTOR PEREIRA. RM 366441

RHÔMULO MOURÃO CAITANO DOS SANTOS. RM 365205

SUMÁRIO

1. Introdução
2. Objetivo
3. Contexto do problema
4. Exploração e Análise dos dados
5. Pré-processamento
6. Feature Engineering
7. Treinamento do modelo
8. Modelos e Resultados
9. Painel e Predição - Streamlit
10. Apêndice Técnico

INTRODUÇÃO

- A obesidade é uma condição de saúde pública que afeta milhões de pessoas no mundo.
- O diagnóstico e o acompanhamento exigem avaliação de múltiplos fatores: dados antropométricos (peso, altura, IMC), histórico familiar e hábitos de vida (alimentação, atividade física, transporte, uso de dispositivos eletrônicos).
- A combinação de *Machine Learning* e visualização de dados permite criar ferramentas que apoiam a equipe médica na triagem e na tomada de decisão, sem substituir o julgamento clínico.
- Este projeto desenvolveu um pipeline de *Machine Learning* para classificação de níveis de obesidade e um painel interativo em Streamlit que integra predição, análises e insights personalizados.

OBJETIVO

Desenvolver um sistema de apoio ao diagnóstico de obesidade que:

1

Classifique o nível de obesidade do paciente (abaixo do peso, normal, sobrepeso I/II, obesidade I/II/III) com assertividade acima de 75%.

2

Ofereça predições por modelos distintos (Random Forest e Regressão Logística), para maior confiabilidade.

3

Disponibilize um painel analítico com visualizações e insights dinâmicos conforme filtros de faixa etária e gênero.

4

Forneça dicas personalizadas ao paciente com base na predição e no perfil (atividade física, transporte, alimentação, etc.).

CONTEXTO DO PROBLEMA

- Contexto:

A obesidade é uma condição de saúde que cresce em escala mundial, totalmente ligada a diminuição da qualidade de vida e o aumento de doenças crônicas

- Problema:

Profissionais da saúde enfrentam dificuldades em identificar o nível de obesidade, já que são diversos fatores envolvidos, desde hábitos alimentarem e rotina de atividades fisicas até aspectos genéticos e sociais

EXPLORAÇÃO E ANÁLISE DOS DADOS

- Nesta etapa, avaliamos a estrutura, a completude e a qualidade do dataset antes do modelamento.
- Estrutura: O dataset possui 2.111 registros e 17 variáveis: 8 numéricas (idade, altura, peso, FCVC, NCP, CH2O, FAF, TUE) e 9 categóricas (gênero, histórico familiar, FAVC, CAEC, SMOKE, SCC, CALC, MTRANS e a variável alvo Obesity).
- Completude: Não há valores nulos em nenhuma coluna; o dataset está completo para modelagem.
- Qualidade: As variáveis de escala (FCVC, NCP, CH2O, FAF, TUE) podem apresentar decimais no arquivo original; serão arredondadas e limitadas aos intervalos definidos no dicionário na etapa de Feature Engineering. Não há inconsistências evidentes nos domínios das variáveis categóricas.

2. Exploração e qualidade dos dados

```python
df.info()
print('\nValores nulos:')
print(df.isnull().sum())
print('\nDistribuição da variável alvo (Obesity):')
print(df['Obesity'].value_counts())
```

✓ 0.0s

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2111 entries, 0 to 2110
Data columns (total 17 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Gender          2111 non-null   object
 1   Age             2111 non-null   float64
 2   Height          2111 non-null   float64
 3   Weight          2111 non-null   float64
 4   family_history  2111 non-null   object
 5   FAVC            2111 non-null   object
 6   FCVC            2111 non-null   float64
 7   NCP             2111 non-null   float64
 8   CAEC            2111 non-null   object
 9   SMOKE           2111 non-null   object
 10  CH2O            2111 non-null   float64
 11  SCC             2111 non-null   object
 12  FAF             2111 non-null   float64
 13  TUE             2111 non-null   float64
 14  CALC            2111 non-null   object
 15  MTRANS          2111 non-null   object
 16  Obesity         2111 non-null   object
dtypes: float64(8), object(9)
memory usage: 280.5+ KB
...
Overweight_Level_II    290
Normal_Weight          287
Insufficient_Weight    272
Name: count, dtype: int64
```

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```python
# df.describe(include='all').T
df.isnull().sum()
```

[33] ✓ 0.0s

```
Gender            0
Age               0
Height            0
Weight            0
family_history    0
FAVC              0
FCVC              0
NCP               0
CAEC              0
SMOKE             0
CH2O              0
SCC               0
FAF               0
TUE               0
CALC              0
MTRANS            0
Obesity           0
```

PRÉ-PROCESSAMENTO

4. Pré-processamento para o modelo

- Separar features numéricas e categóricas.
- Codificar categóricas (One-Hot para não ordinais).
- Padronizar numéricas (StandardScaler).
- Codificar o target (LabelEncoder) para classes de obesidade.

```python
# Variável alvo
target_col = 'Obesity'
y = df[target_col]

# Ordinal para CAEC e CALC: no=0, Sometimes=1, Frequently=2, Always=3
ordinal_map = {'no': 0, 'Sometimes': 1, 'Frequently': 2, 'Always': 3}

# Numéricas (incluindo CAEC e CALC codificados como ordinal)
num_cols = ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE', 'CAEC', 'CALC']
X_num = df[['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE']].copy()
X_num['CAEC'] = df['CAEC'].map(ordinal_map).fillna(0).astype(int)
X_num['CALC'] = df['CALC'].map(ordinal_map).fillna(0).astype(int)

# Categóricas (apenas as que continuam One-Hot)
cat_cols = ['Gender', 'family_history', 'FAVC', 'SMOKE', 'SCC', 'MTRANS']
X_cat = df[cat_cols].copy()
X_cat_encoded = pd.get_dummies(X_cat, drop_first=True, dtype=int)

X = pd.concat([X_num.reset_index(drop=True), X_cat_encoded.reset_index(drop=True)], axis=1)
print('Shape das features:', X.shape)
print('Colunas:', list[Any](X.columns))
```

✓ 0.0s Python

```
Shape das features: (2111, 19)
Colunas: ['Age', 'Height', 'Weight', 'FCVC', 'NCP', 'CH2O', 'FAF', 'TUE', 'CAEC', 'CALC', 'Gender_Male', 'family_history_yes', 'FAVC_yes', 'SMOKE_yes', 'SCC_yes', 'MTRANS_Bike', 'MTRANS_Motorbike', 'MTRANS_Public_Transportation', 'MTRANS_Walking']
```

```python
# LabelEncoder para o target (preservar mapeamento para interpretação)
le_target = LabelEncoder()
y_encoded = le_target.fit_transform(y)
class_names = le_target.classes_
print('Classes do target:', class_names)
```

✓ 0.0s Python

```
Classes do target: ['Insufficient_Weight' 'Normal_Weight' 'Obesity_Type_I' 'Obesity_Type_II'
 'Obesity_Type_III' 'Overweight_Level_I' 'Overweight_Level_II']
```

```python
# Divisão treino / teste (estratificada para manter proporção das classes)
X_train, X_test, y_train, y_test = train_test_split(
    X, y_encoded, test_size=0.25, random_state=42, stratify=y_encoded
)

# Padronização das features numéricas (apenas nas numéricas para não afetar dummies)
scaler = StandardScaler()
num_idx = [X.columns.get_loc(c) for c in num_cols if c in X.columns]
X_train_std = X_train.copy()
X_test_std = X_test.copy()
X_train_std.iloc[:, num_idx] = scaler.fit_transform(X_train[num_cols])
X_test_std.iloc[:, num_idx] = scaler.transform(X_test[num_cols])

print('Treino:', X_train_std.shape[0], '| Teste:', X_test_std.shape[0])
```

✓ 0.0s Python

```
Treino: 1583 | Teste: 528
```

- As variáveis numéricas (idade, altura, peso, escalas) permanecem como estão; CAEC e CALC são codificadas de forma*ordinal (no → 0, Sometimes → 1, Frequently → 2, Always → 3) por terem ordem natural. As demais categóricas (gênero, histórico familiar, FAVC, SMOKE, SCC, MTRANS) passam por One-Hot Encoding com drop_first para evitar multicolinearidade. Ao final, o conjunto de features possui 19 colunas.

- Em seguida, fazemos train_test_split estratificado (75% treino, 25% teste) para manter a proporção das classes, aplicamos StandardScaler nas colunas numéricas (sem padronizar as dummies) e usamos LabelEncoder no target para converter as 7 classes de obesidade em inteiros. Os dados ficam prontos para o treinamento.

FEATURE ENGINEERING

3. Feature Engineering

Conforme o dicionário e boas práticas:

- **IMC (opcional):** nesta versão não criamos BMI; o modelo usa Height, Weight e as demais features.
- **Arredondamento de escalas:** FCVC (1–3), NCP (1–4), CH2O (1–3), FAF (0–3), TUE (0–2).
- **Codificação de variáveis categóricas** para o modelo.

```python
def feature_engineering(df):
    df = df.copy()

    # Sem BMI — usamos apenas Height e Weight como numéricas
    # Arredondar variáveis de escala (conforme dicionário)
    scale_cols = {'FCVC': (1, 3), 'NCP': (1, 4), 'CH2O': (1, 3), 'FAF': (0, 3), 'TUE': (0, 2)}
    for col, (low, high) in scale_cols.items():
        if col in df.columns:
            df[col] = np.clip(np.round(df[col].astype(float)), low, high).astype(int)

    return df

df = feature_engineering(df)
print('Novas colunas após feature engineering:')
orig_cols = ['Gender','Age','Height','Weight','family_history','FAVC','FCVC','NCP','CAEC','SMOKE','CH2O','SCC','FAF','TUE','CALC','MTRANS','Obesity']
print([c for c in df.columns if c not in orig_cols])
df[['Height', 'Weight']].head()
```

✓ 0.0s

```
Novas colunas após feature engineering:
[]
```

	Height	Weight
0	1.62	64.0
1	1.52	56.0
2	1.80	77.0
3	1.80	87.0
4	1.78	89.8

O modelo utiliza Height e Weight diretamente como features numéricas, sem criar a variável IMC. As variáveis de escala FCVC, NCP, CH2O, FAF e TUE vêm no dataset com decimais; fazemos arredondamento e limitamos aos intervalos definidos no dicionário (por exemplo, FCVC 1–3, FAF 0–3) para remover ruído e garantir valores inteiros válidos.

TREINAMENTO DO MODELO

Processo:

Foram testados dois modelos Regressão Logística e Random Forest, com o objetivo de identificar qual teria melhor desempenho na previsão dos níveis de obesidade

Resultado:

Regressão Logística: 88,07% de acurácia

Random Forest: 93,75% de acurácia

5. Treinamento do modelo

Treinamos **Random Forest** e **Regressão Logística** e escolhemos o que tiver melhor desempenho (meta: acurácia > 75%).

```python
models = {
    'Regressão Logística': LogisticRegression(max_iter=1000, random_state=42),
    'Random Forest': RandomForestClassifier(n_estimators=200, max_depth=15, min_samples_leaf=2, random_state=42),
}

results = {}
for name, model in models.items():
    model.fit(X_train_std, y_train)
    y_pred = model.predict(X_test_std)
    acc = accuracy_score(y_test, y_pred)
    results[name] = {'model': model, 'accuracy': acc, 'predictions': y_pred}
    print(f'{name}: Acurácia = {acc:.2%}')
```

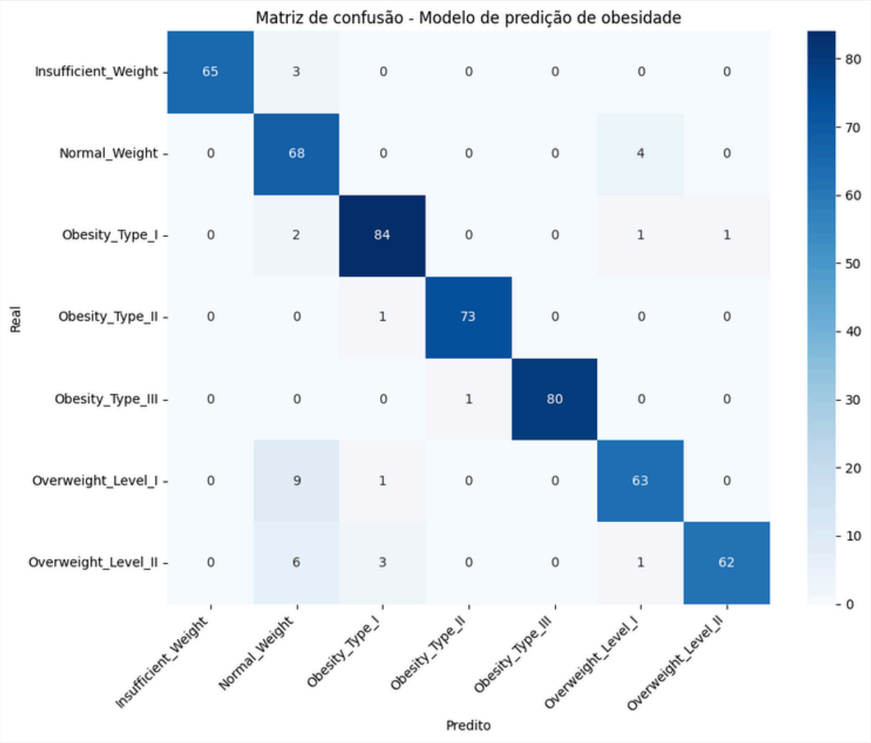
✓ 0.3s

```
Regressão Logística: Acurácia = 88.07%
Random Forest: Acurácia = 93.75%
```

```python
# Escolher o modelo com maior acurácia
best_name = max(results, key=lambda k: results[k]['accuracy'])
best_model = results[best_name]['model']
best_pred = results[best_name]['predictions']
print(f'\nModelo selecionado: {best_name}')
print(f'Acurácia no conjunto de teste: {results[best_name]["accuracy"]:.2%}')
```

✓ 0.0s

```
Modelo selecionado: Random Forest
Acurácia no conjunto de teste: 93.75%
```



6. Avaliação detalhada (assertividade > 75%)

```python
print('Relatório de classificação (conjunto de teste):')
print(classification_report(y_test, best_pred, target_names=class_names))
print('F1-Score (macro):', round(f1_score(y_test, best_pred, average='macro'), 4))
```

[40] ✓ 0.0s

```
Relatório de classificação (conjunto de teste):
                     precision    recall  f1-score   support

Insufficient_Weight       1.00      0.96      0.98        68
      Normal_Weight       0.77      0.94      0.85        72
     Obesity_Type_I       0.94      0.95      0.95        88
    Obesity_Type_II       0.99      0.99      0.99        74
   Obesity_Type_III       1.00      0.99      0.99        81
 Overweight_Level_I       0.91      0.86      0.89        73
Overweight_Level_II       0.98      0.86      0.92        72

           accuracy                           0.94       528
          macro avg       0.94      0.94      0.94       528
       weighted avg       0.94      0.94      0.94       528

F1-Score (macro): 0.9375
```

MODELOS E RESULTADOS

- O Random Forest foi selecionado para integrar o sistema final, garantindo previsões mais precisas e consistentes

- Para avaliar o desempenho além da acurácia, utilizamos o relatório de classificação (precision, recall e F1-score por classe) e o F1-Score macro, que equilibra o desempenho entre as sete classes de obesidade. Isso é importante porque classes com menos amostras podem ser negligenciadas pela acurácia global, e o F1 considera tanto os acertos quanto os erros de cada classe.

- A matriz de confusão mostra, em formato visual, quantas previsões foram corretas (diagonal) e onde ocorreram erros (fora da diagonal). Assim, verificamos se o modelo confunde classes vizinhas (por exemplo, Sobrepeso I com Sobrepeso II) e se a assertividade atinge a meta de 75% no conjunto de teste.

PAINEL ANALÍTICO E PREDIÇÃO - STREAMLIT

CONCLUSÃO

O sistema desenvolvido mostrou-se eficaz em transformar dados demográficos e comportamentais em informações úteis para apoiar diagnósticos de obesidade.

O modelo Random Forest apresentou alta precisão (93,75%) e foi integrado a um painel interativo no Streamlit, complementado por análises no Power BI. Essa combinação oferece uma solução prática e acessível, capaz de auxiliar profissionais da saúde em decisões preventivas e estratégicas.

APÊNDICE TÉCNICO

- Painel no Streamlit
- Dashboard em PowerBI
- Notebook (.ipynb) com o pipeline

(Notebook .ipynb encontram-se na pasta zipada junto a este documento)